Neural Differential Equations for Distribution Mapping

Curtis Mellsop
Victoria University of Wellington

I. Introduction

This report investigates the use of neural-network-based differential equations to map samples from one distribution to another. It implements both deterministic Ordinary Differential Equations (ODEs) and Stochastic Score-based Differential Equations (SDEs) to transform source distributions into a target distribution. The performance and behavior of these models are analyzed through visualizations of sample trajectories and final distributions, highlighting differences in convergence, accuracy, and stochasticity.

II. Theory

A. Ordinary Differential Equation (ODE) Approach

The ODE approach defines a deterministic flow that learns a direct, non-random path from a source distribution to a target distribution.

a) Neural Network and Velocity Field: The evolution of each sample is described by the first-order ODE:

$$\frac{d\mathbf{x}}{dt} = \mathbf{f}_\theta(\mathbf{x}(t), t), \tag{1}$$

where $\mathbf{x}(t) \in \mathbb{R}^2$ represents the sample at time t and $\mathbf{f}_\theta : \mathbb{R}^2 \times [0,1] \to \mathbb{R}^2$ is a neural network predicting the velocity field. Specifically, $\mathbf{f}_\theta(x, t)$ provides the instantaneous direction and magnitude that the sample should follow at time t, guiding it along a smooth trajectory from the source distribution toward the target distribution.

b) Linear Interpolant Approach: The model was trained using the linear interpolant approach, in which intermediate points are defined as

$$x_t = (1-t)x_0 + ty, \tag{2}$$

where x_0 is a sample from the source distribution and y is a sample from the target distribution. The neural network $\mathbf{f}_\theta$ was trained to match the corresponding velocity

$$v_\theta(x_t, t) \approx \frac{y - x_t}{1-t}, \tag{3}$$

by minimizing the mean-squared error between the predicted and target velocities.

c) Euler Method: Once trained, samples are mapped from the source to the target distribution by integrating the ODE using Euler's method. At each timestep Δt, the sample is updated according to

$$\mathbf{x}_{t-\Delta t} = \mathbf{x}_t - \mathbf{f}_\theta(\mathbf{x}_t, t)\,\Delta t, \tag{4}$$

where $\mathbf{f}_\theta(\mathbf{x}_t, t)$ is the predicted velocity. Repeating this update iteratively produces a smooth, deterministic trajectory for each sample from the initial distribution toward the target distribution. The step size Δt can be chosen to balance integration accuracy and computational cost.

B. Stochastic Differential Equation (SDE) Approach

The SDE approach defines a stochastic flow that gradually transforms samples from a simple distribution to a target distribution by incorporating both a deterministic drift and a stochastic diffusion term.



Fig. 1: Forward Process of SDE training.

a) Forward Process: The forward SDE gradually adds noise to samples, transforming them from the target distribution towards the known source distribution (Fig. 1). The formula is

$$d\mathbf{x} = f(\mathbf{x}, t)\,dt + g(t)\,d\mathbf{W}_t, \tag{5}$$

where $f(\mathbf{x}, t)$ is the drift term (often set to zero), $g(t)$ is the diffusion coefficient, and $d\mathbf{W}_t$ is the Wiener process. This process defines a series of noisy distributions $p_t(\mathbf{x})$ used to train the model.

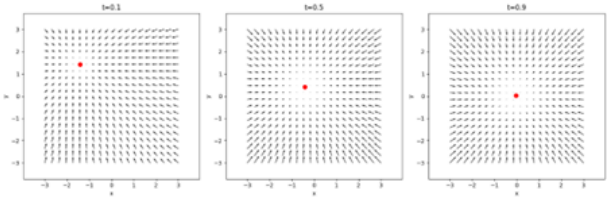


Fig. 2: Score directions at times t.

b) Neural Network and Score Function: A neural network $\mathbf{s}_\theta(\mathbf{x}, t)$ is trained to approximate the score function, the gradient of the log-density:

$$\mathbf{s}_\theta(\mathbf{x}, t) \approx \nabla_{\mathbf{x}} \log p_t(\mathbf{x}). \tag{6}$$

The score indicates the direction toward regions of higher probability at each timestep (Fig. 2), guiding the reverse transformation from the source to the target distribution. The network is trained with an MSE loss between the predicted and true scores computed from the forward process.

c) Euler–Maruyama Integration: Once trained, samples are generated by integrating the reverse-time SDE using the Euler–Maruyama method:

$$\mathbf{x}_{t-\Delta t} = \mathbf{x}_t + \left[f(\mathbf{x}_t, t) - g(t)^2 \mathbf{s}_\theta(\mathbf{x}_t, t)\right]\Delta t + g(t)\sqrt{\Delta t}\,\boldsymbol{\epsilon} \quad (7)$$

This update combines a drift term (guided by the score) and a stochastic term (noise), producing diverse trajectories that converge to the target distribution. The step size Δt controls the integration accuracy and computational cost.

III. Experiments

This section presents the experimental setup and results for mapping samples between source and target distributions using ODE and SDE models.

A. Setup

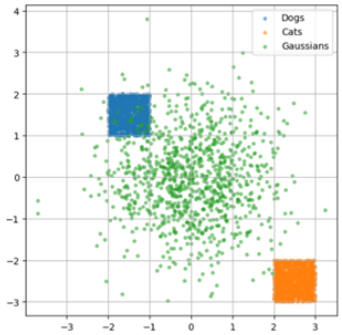


Fig. 3: Input data.

a) Source/Target Distributions: There were three types of distributions: Dogs, Cats, and Gaussians (Fig. 3). Dogs and Cats were sampled uniformly from 2D squares, with corners at $(-1, 1)$ and $(-2, 2)$ for Dogs, and $(2, -2)$ and $(3, -3)$ for Cats. Gaussian samples were drawn from a 2D normal distribution with mean $\mathbf{0}$ and identity covariance scaled by a factor s.

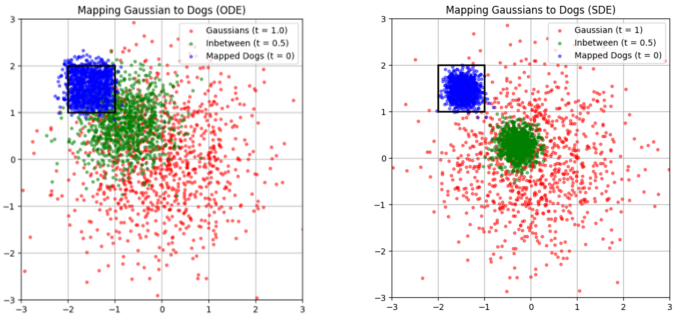
b) Neural Network Models: All models shared the same architecture, consisting of an input layer of size 3, a hidden layer with 128 units, and an output layer of size 2. The input vector $[x, y, t]$ represents the coordinates at a given time, while the output corresponds to either the predicted velocity field (for the ODE models) or the score function (for the SDE model).

c) Hyperparameters: All models were trained over 5000 epochs with a batch size of 128, a learning rate of 0.001, and the Adam optimizer. 100 steps were used to produce the target distribution from the source.

B. Comparison of the ODE and SDE Approaches

The models were compared visually, assessing both the final distributions and the paths taken to reach them. The comparison was qualitative, focusing on differences in sample trajectories and how closely each approach reproduced the target distribution.

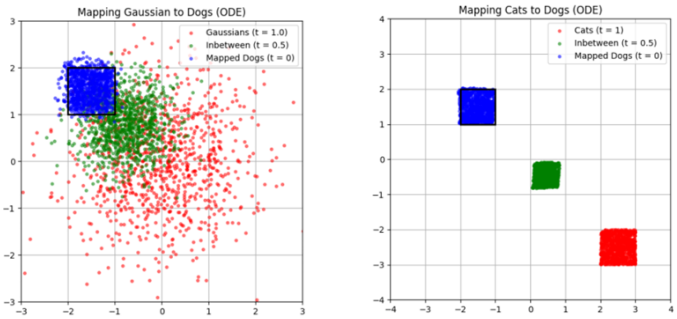
The ODE model produced smooth, direct trajectories toward the target, with a final distribution that largely matched the target but included some outliers in the top-left. In contrast, the SDE model initially condensed samples near the center before spreading them, with all points remaining within the target square while retaining circular patterns reminiscent of the Gaussian source.



(a) ODE: Gaussian → Dog (b) SDE: Gaussian → Dog

Fig. 4: ODE and SDE sample trajectories.

C. Comparison of Two ODE Approaches



(a) ODE: Gaussian → Dog (b) ODE: Cat → Dog

Fig. 5: Different source sample trajectories.

This experiment investigates how the choice of source distribution influences the predicted mapping to the target distribution.

When the source distribution shares the same shape as the target, the mapping more accurately reproduces the target. Interestingly, unlike the Gaussian-to-Dog mapping, this mapping is not direct; the points slightly condense near the center before spreading to the target distribution.

IV. Conclusion

This study explored ODE and SDE models for mapping samples between distributions. ODEs produced smooth, direct trajectories with minor outliers, while SDEs followed stochastic paths with points fully contained within the target distribution, though they retained some source-like patterns. Experiments with different source distributions showed that mappings are more accurate when the source shape matches the target, even if trajectories initially condense before spreading. Overall, these results highlight the trade-offs between deterministic and stochastic approaches in generative modeling, providing insights into their behavior and accuracy.

STATEMENT

This report and the code provided were entirely my own work.

Tools used: Overleaf, Google Colab, PyTorch, NumPy, Matplotlib.

Source code is available from the following repository: https://github.com/CRM48/victoria_projects/tree/main/neural_networks/neural_differential_equations.

It contains the ipynb files which can be opened in Colab by clicking the link on the file.